

The main question:

- 1. ให้นิเทศเสนอการพัฒนาโปรแกรม **network application** โดยเขียนวัตถุประสงค์ของโปรแกรม ว่าเป็นโปรแกรมอะไร ใช้สำหรับทำอะไร **application** ที่พัฒนามีคุณลักษณะ (characteristics) อย่างไร ต้องการใช้ **service model** ใดของ **Transport Layer** (UDP หรือ TCP) เพราะอะไร อธิบาย จากนั้นให้ ออกแบบ **Application-Layer Protocol** ที่ประกอบด้วย **request and response messages** ที่ใช้ในการสื่อสาร ระหว่าง **Client** และ **Server** (หรือ **Peers**) ของ **network application** ที่นิสิตจะพัฒนา ให้ตั้งชื่อ **protocol** ของนิสิต ส่งไฟล์ pdf ที่อธิบาย **protocol** ที่นิสิตออกแบบ และตอบคำถามทั้งหมดในข้อ 1 โดยละเอียด
-

Answers

1. What is my network application?

- A two players turn-based RPG game (1 VS 1) that implements custom Application-layer protocol DUEL/1.0 running on TCP

2. What is it for?

- To help explain how the application layer protocol can be design. what kind of transport layer should be implement in this kind of application, explain with a valid reason from what I've learnt. Showing all of these component via a turn-based RPG game.

3. Application Characteristics

- **Client-server**, not peer-to-peer — the server holds the single source of truth for HP, mana, and active status effects; clients never trust each other's local state.
- **Stateful / session-based** — a connection moves through distinct phases (handshake → lobby/ready → combat → game over), and the server has to track exactly where each client is in that lifecycle.
- **Turn-based, not real-time** — exactly one player acts per turn; there's no strict timing constraint on how fast a message must arrive.
- **Text-based, human-readable protocol** — every message is plain text, not binary, so it can be logged, printed, and demoed directly.

- **Synchronous request/response** — the server always replies to a request before the next one is expected, and further broadcasts the result to the *other* client too, so both sides stay synchronized after every action.

4. What service model of transport layer did I use

- **TCP** because based on the design/application characteristics, we should prioritize reliability, ordered data more than raw speed.
 - **Reliability** — a lost `ACTION` message would desync the match (one player thinks it's still their turn; the other is stuck waiting). Turn-based state cannot tolerate silent packet loss the way a real-time video/audio stream can.
 - **Ordering** — status-effect ticks, action resolution, and turn-swaps must happen in the exact sequence they were generated. UDP gives no such guarantee.
 - **Connection state for free** — the lobby handshake and forfeit/disconnect detection map naturally onto a persistent TCP connection; the server can tell a player left just from the socket closing, with no extra protocol needed.

5. Protocol Design

- DUEL/1.0 (Protocol's name)
 - modeled in HTTP style for readability, easy to debug
- *Requests (client → server)* — one line, `VERB arg1 arg2 ... :`

VERB	EXAMPLE	MEANING
HELLO	HELLO Chino	Identify yourself on connect
CHOOSE	CHOOSE MAGE	pick a class and its various (if existed)
READY	READY	a signal to start the match
ACTION	ACTION SPELL THUNDER SHOCK	perform an action in combat
BLOCK	BLOCK	defensive action
QUIT	QUIT	forfeit

- *Responses (server → client)* — a status line plus headers, blank-line terminated:

```
DUEL/1.0 <code> <phrase>
Key: Value
...
```

- *Status codes*: 1xx informational (waiting), 2xx success (`200 OK` , `201 MATCH_STARTED` , `202 TURN_RESOLVED` , `210 GAME_OVER`), 4xx client error (`400 BAD_REQUEST` , `403`

INVALID_ACTION), 5xx server error — full table in `protocol.py` .

Chinoros Kerdtasom 6710450694